Lesson 5: Enemy Ranks

Unity

VIDEO GAME CREATION

This week: one enemy becomes many. Your job is to turn last week's single enemy into a whole row of them, using a loop instead of repeating yourself by hand.

Open your own [YourName]Game project from Lesson 4 - don't start a new one. Everything this week builds on the enemy and collision detection you already have.

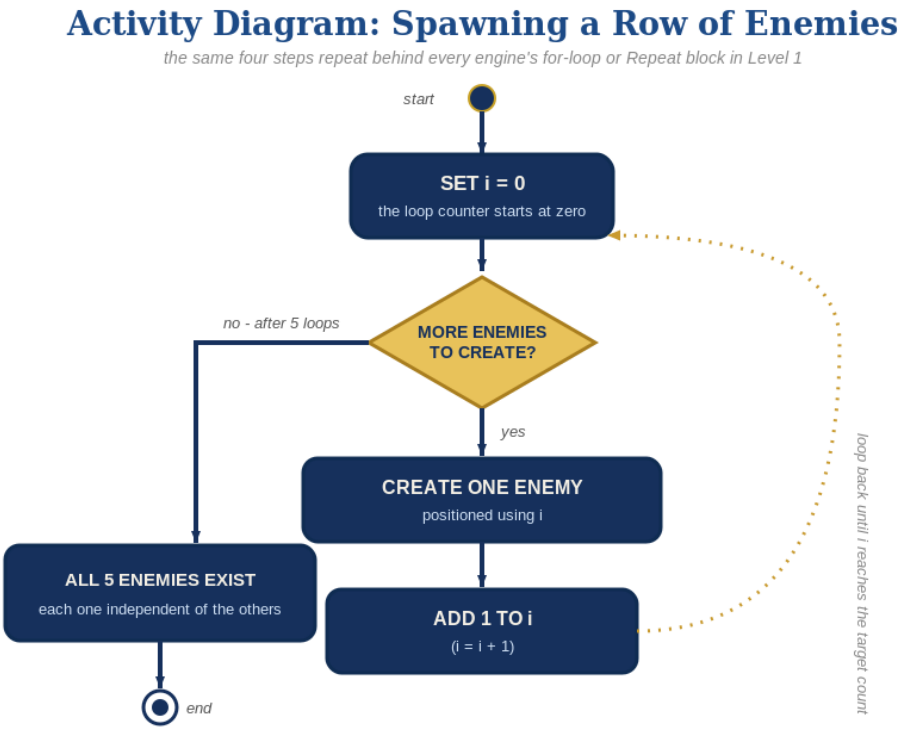
Tip: no new assets this week - you're reusing bullet_yellow.png, sfx_shoot.wav, enemy_invader_red.png, and sfx_hit.wav from your Lesson 3 and 4 Assets folders.

Setup

1. Open your [YourName]Game project from Lesson 4.
2. Delete the single Enemy prefab instance you manually placed in your scene last week.
3. Create a new empty GameObject named EnemySpawner. Create a new C# script named EnemySpawner, attach it to this new GameObject.

Level 1 - Minimal (MVP)

Several enemies sit on screen in a row, created by a loop instead of placed or spawned one at a time. Shooting any one of them destroys just that one. This is a complete finish for today - reaching this is enough to move on to next week.



Every engine's Level 1 solution runs this same shape - a counter, a repeat check, one creation step, and an increment - even though the exact blocks or code syntax differ. Tracing a spawn bug means finding which of these four steps broke.

The spawn loop, in general - the same counter, repeat check, creation step, and increment behind Level 1 in every engine, whatever the exact blocks or syntax look like.

4. In EnemySpawner.cs, add a public GameObject enemyPrefab field and an int enemyCount, then use a for loop in Start() to Instantiate() several copies at different positions (code below).
5. In the Inspector, drag your Enemy prefab onto EnemySpawner's Enemy Prefab field.
6. Press Play - you should see a row of enemies, and shooting any one of them should destroy just that one.

```
// EnemySpawner.cs:
public GameObject enemyPrefab;
public int enemyCount = 5;

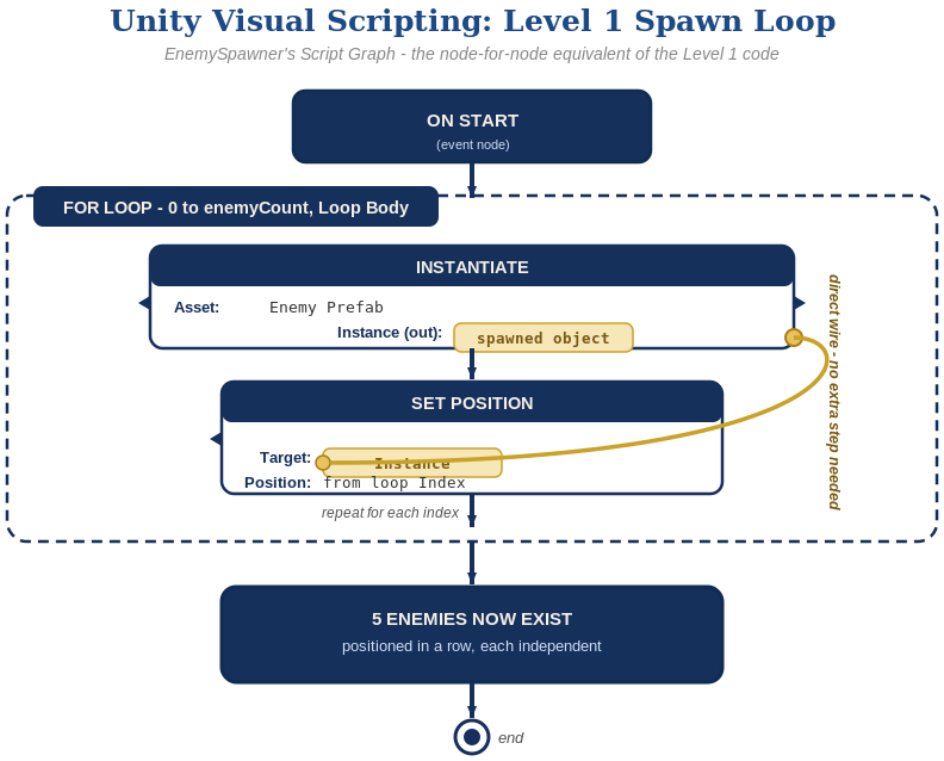
void Start() {
    for (int enemyIndex = 0; enemyIndex < enemyCount; enemyIndex++) {
        Vector3 pos = new Vector3(-4f + enemyIndex * 2f, 4f, 0f);
        Instantiate(enemyPrefab, pos, Quaternion.identity);
    }
}
```

Alternate Path: Visual Scripting

The same for-loop spawn pattern works in a Script Graph instead of C#.

7. On the EnemySpawner GameObject, add a Script Machine component and open a new Script Graph.

8. Add an On Start event node feeding a "For Loop" node (0 to enemyCount), with an "Instantiate" node inside it (Enemy prefab as the asset input) feeding a "Set Position" node using the loop's current index.
9. Set up the Enemy Prefab field the same way as the C# path.



Instantiate's own output pin already holds the exact instance it just created - dragging that pin straight into Set Position's Target is the normal way node graphs work, not a special trick this lesson had to reach for.

The Script Graph, node by node - Instantiate's own output pin plugs straight into Set Position's Target, no extra step needed.

Level 2/3 already work the same way as the C# path, since the per-instance logic lives on the Enemy and Bullet prefabs' own graphs, not the spawner's.

Level 2 - Complete (Alpha/Beta)

Every enemy in the row can be individually shot and destroyed without affecting the others, and a bullet that misses every enemy still behaves exactly like it did last week.

- Shoot several different enemies in a row, including some misses - each hit destroys exactly the one enemy that was actually touched, and every other enemy is unaffected.
- This already works with no extra code: Bullet.cs's OnTriggerEnter2D and Enemy.cs's own Start() are self-contained per instance, so every prefab copy the loop creates behaves independently and correctly, exactly the way Lesson 4 built it.

Level 3 - Refined (Ready to Publish) - pick any that interest you

- Sound: this already works with no changes - AudioSource.PlayClipAtPoint() in Bullet.cs runs independently for every enemy that gets hit.

- Visual: vary each enemy's SpriteRenderer.color based on the loop index enemyIndex at spawn time, for a striped row effect - this needs a small public setter method or a spawner-side GetComponent<SpriteRenderer>() call right after Instantiate(). Alternate new Color(0.8f, 0.1f, 0.2f), a deep crimson red, with new Color(0.95f, 0.4f, 0.5f), a lighter dusty rose/pink (Unity's Color values run 0-1 instead of 0-255, so 0.8f is roughly 200 out of 255) - the result is every other enemy showing up as a visibly different shade of red, not a smoothly blended color.
- Structure: keep EnemySpawner.cs's only job being creating and positioning enemies - resist the urge to also handle collision-response logic there; that still belongs entirely inside Enemy.cs and Bullet.cs.

If Something's Wrong

- Only one enemy appears, or they're all stacked in the same spot: confirm the loop variable enemyIndex is actually used in the position calculation, not a fixed Vector3.
- Console error about a missing prefab reference: confirm the Enemy Prefab field on EnemySpawner is actually assigned in the Inspector.
- Old and new enemies both appear: confirm the manually-placed Lesson 4 Enemy instance was actually deleted from the Hierarchy.

Want More?

Student Resource List > Unity section, or learn.unity.com